

KIRIROM INSTITUTE OF TECHNOLOGY

Department of Engineering — Bachelor in Software Engineering

Batch 11 Final-Year Software Project

REVIEW 1

Requirements, Design and Prototype Package

Property Rental System

A web-based rental platform with listing management, online booking requests, payment integration (planned), and AI-powered recommendations (planned)

Field	Detail
Group ID	B11-G06
Project short title	PropertyRentalSystem
Student 1	Sopheak Rithmunny — 2702230032 — rithmunny.sopheak22@kit.edu.kh
Student 2	Lim Phanith — 2702230014 — phanith.lim22@kit.edu.kh
Supervisor	Mr. Leo
Repository	https://github.com/Rithmunny/Property-Rental-System.git
Review 1 date	27 August 2026
Submission date	20 August 2026
Planned completion	7 November 2026

File: B11-G06_PropertyRentalSystem_Review1_Document.pdf

This package accompanies the Review 1 presentation and the live proof of concept.

Contents

1. Cover page
2. Response to Review 0 feedback and post-approval corrections
3. Corrected proposal summary
4. Expanded literature and existing-system analysis
5. Requirements-elicitation evidence
6. Functional requirements
7. Non-functional requirements
8. User roles, permissions and user stories
9. Use cases and process models
10. System architecture
11. Database design
12. Detailed design diagrams
13. UI/UX design
14. API and integration design
15. Security and privacy design
16. Technology setup and repository
17. Updated work plan and risk register
18. Updated thesis progress (Chapters 1–4)
19. Prototype evidence
20. References and appendices

1. Cover Page

The cover at the front of this file is the official title page. Group identity, student IDs, supervisor, repository and dates are repeated here so that the document remains identifiable if pages are printed separately.

Table 1.1 — Document control

Item	Value
Document type	Review 1 Requirements and Design Package (PDF)
Matches presentation	B11-G06_PropertyRentalSystem_Review1_Presentation.pptx / .pdf
Approved proposal	B11-P06_PropertyRentalSystem_Proposal.pdf (Review 0, 14 July 2026)
Terminology	tenant, landlord, administrator; listing; rental request; contract; payment (sandbox later)
Referencing	APA 7th edition

2. Response to Review 0 Feedback and Post-Approval Corrections

Review 0 (15 July 2026) approved the problem, aim and two-student split: Rithmunny leads the frontend/product surface; Phanith leads database, authentication and integrations. Written panel minutes were not stored in the project repository at the time this package was compiled. Table 2.1 therefore records the corrections the group has applied against (a) the official Review 0 guideline checks and (b) gaps found while expanding the approved proposal into a Review 1 design. If the supervisor issues numbered comment IDs, those rows can be added without changing the technical content below.

Table 2.1 — Corrections after Review 0

No.	Comment / required improvement	Action taken	Owner	Evidence in this package
R0-01	The proposal must remain more than basic CRUD and keep booking, payment and AI in a realistic order.	Core path is listing + search + Request to Rent + accept/decline with a contract stub. Payment sandbox and Gemini/OpenAI stay after Review 1, matching the original Gantt (core first).	Both	§3, FR table, §17
R0-02	Elicitation must be evidenced, not only informal remarks.	Observation record, informal-discussion themes, market sources, and a questionnaire instrument were added. Findings are traced to FR IDs.	Rithmunny	§5, Appendix A
R0-03	Technology must be feasible for the group and justified.	Frontend is Vite + React 19 (already implemented SPA) instead of Next.js. PostgreSQL and Auth remain on Supabase as proposed. The change is documented, not hidden.	Both	§3, §10, §16
R0-04	Work plan must hit official review dates and finish by 7 November 2026.	Review 1 is 27 August 2026 (student guidelines), not “late September”. Later reviews follow the handbook. Gantt and risks were rewritten.	Both	§17
R0-05	Both students must understand the whole system and show repository work.	Responsibility matrix, module map, and a requirement that Phanith commits schema/RLS from his GitHub account.	Phanith / Rithmunny	§16, Fig. 12.4
R0-06	Users, roles and security must be explicit.	RBAC matrix, RLS design, no client-chosen admin role in the real adapter, validation and data-minimisation notes.	Phanith	§8, §15
R0-07	Literature and comparable systems must show a clear gap.	Comparison of Realestate.com.kh, Facebook/Telegram, and Booking-style platforms; gap restated as listing + request + payment + later AI in one product for Phnom Penh long-term rent.	Rithmunny	§4
R0-08	Prototype cannot be static screens only.	Design and demo script require login → insert rental_requests → landlord accept → tenant sees accepted, with PostgreSQL as the store. UI already implements this flow on the adapter seam.	Both	§9, §19

The Review 0 proposal filename used B11-P06. The official group ID is **B11-G06**. All Review 1 files use G06.

3. Corrected Proposal Summary

3.1 Approved problem

Property owners in Phnom Penh who manage rentals through phone calls, Facebook, Telegram or paper records face delayed replies, inaccurate availability and double allocation of the same unit. Tenants must contact many landlords one by one to compare price, location and facilities. Classified sites list inventory but leave booking, payment and records off-platform (Bamboo Routes, 2026; Realestate.com.kh, 2026).

3.2 Aim

To design and develop a web-based Property Rental System for the Cambodian long-term rental market that centralises listing, online rental requests (booking), role-based dashboards and, later in the semester, sandbox payment and AI-assisted recommendations.

3.3 Objectives (unchanged in intent; delivery dates updated)

- 1. Analyse landlord and tenant needs for listing, requesting/booking and payment.
- 2. Design a PostgreSQL schema (Supabase) for users, listings, requests, contracts, payments and reviews.
- 3. Implement authentication and dashboards for tenant, landlord and administrator.
- 4. Implement a request-to-rent workflow that prevents a second accepted occupancy on the same listing.
- 5. Integrate a payment gateway in sandbox mode (ABA PayWay or Stripe) after the core path is stable.
- 6. Integrate an AI recommendation feature (OpenAI or Gemini) after the core path is stable.
- 7. Test, document and complete the project by 7 November 2026.

3.4 Scope corrections

Table 3.1 — In scope versus Review 1 subset

Semester (Must / Should)	Review 1 proof of concept
Auth, three dashboards, listing CRUD, search/filter, Request to Rent, accept/decline, contract stub, saved homes, admin moderation	Auth + listings from the database + Request to Rent + accept + visible status
Sandbox payment, notifications, reviews, analytics, AI recommendations	Designed (API + schema + sequence); not required as live integrations on 27 August

Out of scope (unchanged): live money, automated legal leases, government title registries, native mobile apps. Property-management staff remain a secondary actor; they are not a fourth login role this semester (Could / Won't).

3.5 Technology correction

The proposal named Next.js. The implemented presentation layer is a **Vite 6 + React 19** single-page application with Tailwind CSS. The team keeps this stack because the public site and three dashboards already exist and the API seam (frontend/src/api) was written for a backend handoff. Database, auth and (later) storage remain **Supabase / PostgreSQL** as approved. Maps already use Leaflet / OpenStreetMap, which the proposal allowed as a free fallback.

3.6 Users

Primary: tenant (renter) and landlord (owner). Stakeholder: administrator. Secondary (not implemented as a role): property-management staff acting on behalf of a landlord.

4. Expanded Literature and Existing-System Analysis

Rental software sits at the intersection of classified advertising, property management systems (PMS) and consumer booking platforms. Sommerville (2016) treats requirements as testable statements traced through design and implementation; this package follows that discipline. Role-based access control is the standard way to separate tenant, landlord and operator privileges in a multi-sided platform (Ferraiolo et al., 2001).

4.1 Comparable systems

Table 4.1 — Critical comparison

System	Strengths	Limitations for this project	Implication
Realestate.com.kh	Large Cambodian inventory; filters by location and type; trusted brand.	No in-product booking, no payment, no tenant–landlord request state machine, no personalised ranking from browsing history.	Discovery exists; the operational gap remains.
Facebook Marketplace / groups and Telegram channels	Zero cost; landlords already present; fast posting.	Unstructured; stale posts; no availability lock; high fraud and double-booking risk; no audit trail.	This is the current process the problem statement attacks.
Airbnb / Booking.com style products	Calendar booking, payments, reviews, host tools.	Optimised for short stay and tourism; fees and KYC are heavy for local long-term rooms; not Khmer-first classified behaviour.	Borrow the request–confirm–pay pattern, not the whole marketplace model.
Generic landlord spreadsheets / paper	Simple for one unit.	No shared availability; poor communication; no tenant self-service.	Dashboard + requests replace this for small landlords.

4.2 Technical literature used in the design

- Backend-as-a-service (Supabase) provides managed PostgreSQL, Auth and Row Level Security, which matches a two-student schedule (Supabase, 2026).
- SPA + BaaS is an accepted architecture when the UI is rich and the data API is relational (MDN Web Docs, 2026).
- Payment platforms (ABA PayWay, Stripe) must stay in sandbox for student projects; live settlement is out of scope (Stripe, 2026).
- LLM recommendation APIs can rank listings from preferences and history, but they need seed volume and a rule-based fallback (OpenAI, 2026). That is why AI is Must for the semester and not the Review 1 demo.

4.3 Project gap

Existing Cambodian channels help a tenant *find* a phone number. They do not give a shared, role-protected record of “this person asked to rent this unit, the owner accepted, the unit is no longer available.” The Property Rental System closes that gap with listings, a rental-request workflow, later sandbox payment, and later AI ranking, which together provide the technical depth Review 0 required (booking-conflict logic, payment workflow, AI integration) without claiming all three are finished on 27 August.

5. Requirements-Elicitation Evidence

Objective 1 required observation and discussion with possible users. Methods used: (1) direct observation of local listing practice; (2) informal discussions with students, workers and people acting as small landlords; (3) domain research on market scale; (4) a structured questionnaire for the remainder of the semester. No personal identifiers are stored in this document.

5.1 Observation (July 2026)

Table 5.1 — Observation record

Item	Record
Setting	Public Facebook property groups, Telegram rental channels, and classified pages used by Phnom Penh renters
What was seen	Listings as photos plus a chat handle; price often in the caption; availability updated by a new post rather than a status field; the same unit sometimes reappeared as “still available” after a claimed booking
Implication	FR-02 (structured listing), FR-04 (single accepted occupancy), FR-07 (notification of status change)

5.2 Informal discussion themes

Table 5.2 — Themes mapped to requirements

Theme	Who	Need	FR / NFR
Too many separate chats to compare rent	Student / worker tenants	Search and filter in one place	FR-03, NFR-03
Landlords reply late or not at all	Tenants	Request with a visible status	FR-04, FR-07
Fear of paying deposits off-platform	Tenants	Recorded payment path (sandbox first)	FR-05, NFR-01
Difficulty tracking who asked about which room	Small landlords	Inbox of requests per listing	FR-02, FR-06
Fake or duplicate ads	Both	Admin moderation	FR-11
Hard to shortlist from many similar condos	Tenants	Recommendations later in the semester	FR-09

5.3 Domain research

Phnom Penh condominium stock has grown toward about 80,000 units, and classified portals already carry tens of thousands of advertisements (Bamboo Routes, 2026; Khmer Times, 2025;

Realestate.com.kh, 2026). Scale supports a software solution; it does not by itself supply booking integrity. That integrity is a product requirement, not a marketing page.

5.4 Questionnaire (instrument)

A 12-item questionnaire (Appendix A) covers search behaviour, willingness to request online, payment trust, and landlord record-keeping. It will be issued to at least eight participants (mix of tenants and landlords) before Review 2. Until those returns are in, design uses Table 5.2 plus the approved proposal evidence. The group will not invent named interview transcripts.

5.5 Key findings used in design

1. The primary tenant job is compare-then-contact; the system must make compare (search) and contact (request) first-class.
2. The primary landlord job is an inbox, not a public blog.
3. Payment and AI add depth but fail if the request state machine is missing — hence Review 1 focus.
4. Admin exists because unmoderated classifieds were observed to be noisy.

6. Functional Requirements

IDs are unique, prioritised with MoSCoW for this semester, and tagged for the Review 1 subset. Each requirement is written so a test can pass or fail.

Table 6.1 — Functional requirements

ID	Requirement	Actor	Semester	Review 1	Trace
FR-01	The system shall allow a user to register with name, email, password and role tenant or landlord, store a password hash, and create a profiles row.	All	Must	Must	UC-01, profiles
FR-01a	The system shall authenticate with email and password and load role from the database, not from a client-supplied admin switch.	All	Must	Must	UC-01, §15
FR-02	The system shall allow a landlord to create, edit and delete their listings with title, type, city, address, price, rooms, area, photos (URL or later storage), amenities and availability.	Landlord	Must	Should	UC-02, properties
FR-03	The system shall allow a tenant (and guests) to search and filter listings by location, price range and property type.	Tenant	Must	Must	UC-03, /rent
FR-04	The system shall allow a logged-in tenant to submit a rent request for an available listing and shall reject a second pending/accepted rent request of the same kind from the same tenant on the same listing.	Tenant	Must	Must	UC-04, rental_requests
FR-04a	When a landlord accepts a rent request, the system shall set that request to accepted, create a contract stub, and set the listing available=false so another tenant cannot obtain a second accepted occupancy.	Landlord	Must	Must	UC-05, Fig. 12.1
FR-04b	The system shall allow a tenant to submit a viewing request with date and time, independent of a rent request.	Tenant	Should	Could	UC-06
FR-05	The system shall process a booking-related payment through ABA PayWay or Stripe in sandbox mode and record status against a contract.	Tenant	Should	Won't (demo)	payments table
FR-06	The system shall present separate dashboards for tenant, landlord and administrator with role-gated routes.	All	Must	Must	RequireRole
FR-07	The system shall notify tenant and landlord when a request is created or its status changes (in-app first; email/Telegram later).	Tenant, landlord	Should	Could	alerts

FR-08	The system shall allow a tenant to submit a rating (1–5) and review text after a booking is accepted or a stay starts.	Tenant	Should	Won't (demo)	reviews
FR-09	The system shall recommend listings using an AI service from saved preferences and browsing history, with a non-AI fallback if the API is down.	Tenant	Must	Won't (demo)	§14
FR-10	The system shall show basic rental analytics (listing count, pending requests, collected rent) for landlords and administrators.	Landlord, admin	Should	Could	dashboards
FR-11	The system shall allow an administrator to list tenants and landlords, change landlord status (pending/active/suspended), and mark a listing rented or available.	Admin	Must	Should	UC-07
FR-12	The system shall allow a tenant to save and unsave listings.	Tenant	Should	Should	saved_properties
FR-13	The system shall allow tenant and landlord to exchange messages linked to a listing or request.	Tenant, landlord	Could	Won't (demo)	messages

Quality check: each FR in Table 6.1 is developer-clear, has a pass/fail test, supports an approved objective, is feasible this semester (or explicitly Won't for Review 1), is prioritised, and traces to a use case or table.

7. Non-Functional Requirements

Table 7.1 — Non-functional requirements (measurable)

ID	Category	Requirement	Measurement
NFR-01	Security	Only authenticated users access dashboards; role mismatch redirects; admin role cannot be self-assigned at register; passwords hashed by Supabase Auth; secrets stay in environment variables.	Access tests: unauthenticated → /login; tenant URL as landlord → landlord home; register as admin rejected.
NFR-02	Performance	Home, /rent and dashboards render within 3 seconds on the demo laptop under test conditions (Wi-Fi, seed data < 5,000 rows).	Stopwatch or DevTools load event, n=5.
NFR-03	Usability	A representative tenant completes search → open listing → Request to Rent with at most one hint from the facilitator.	Task success yes/no; time recorded.
NFR-04	Reliability	After an accepted rent request, a second tenant cannot receive accepted status on the same listing (available=false and/or exclusion constraint).	Two-account test; expected: second accept blocked or listing hidden.
NFR-05	Scalability	Schema (keys, indexes on city, price, landlord_id, tenant_id, status) shall not require redesign at 5,000 listings.	Design review; optional EXPLAIN on search.
NFR-06	Maintainability	Data access is isolated in frontend/src/api with one domain per file; JSON names match the API contract.	Code review against AGENTS.md / API_CONTRACT.md.
NFR-07	Privacy	No card PAN stored; payment references only; screenshots crop emails of real people.	Schema inspection; screenshot checklist.
NFR-08	Availability (prototype)	Demo accounts and seed listings can be reset from SQL seed files before the oral exam.	Restore script run-through.

8. User Roles, Permissions and User Stories

Table 8.1 — Roles and permissions

Action	Guest	Tenant	Landlord	Admin
View public listings / search	Yes	Yes	Yes	Yes
Register	tenant or landlord	—	—	No self-register
Create rent / viewing request	No	Yes	No	No
CRUD listings	No	No	Own rows	Moderate all
Accept / decline requests	No	No	Inbox for own listings	Override if needed
Save listings	No	Yes	No	No
Approve / suspend landlord	No	No	No	Yes
See other users' emails in bulk	No	No	Only applicants on own listings	Yes

8.1 User stories and acceptance criteria (core)

US-01. As a tenant, I want to log in so that I see only the tenant dashboard.

Acceptance: Valid demo tenant credentials open /dashboard/tenant. Invalid password shows an error. Opening /dashboard/landlord redirects away.

US-02. As a tenant, I want to request to rent a listing so that the landlord receives a pending request.

Acceptance: After submit, a rental_requests row exists with status pending. Repeating the same listing shows the existing request rather than a duplicate rent row.

US-03. As a landlord, I want to accept a request so that the tenant sees accepted and the listing is no longer offered as available.

Acceptance: Status accepted; contract stub present; listing available is false; tenant refresh shows accepted.

US-04. As a landlord, I want to add a listing so that tenants can find it in search.

Acceptance: New row with landlord_id = current user; appears on /rent after reload from the database.

US-05. As an admin, I want to suspend a landlord so that the account cannot operate as active.

Acceptance: Status change persists; only admin role can call the update.

US-06. As a tenant, I want to save a home so that I can open it later from Saved Homes.

Acceptance: Toggle writes saved_properties; list matches after logout/login.

9. Use Cases and Process Models

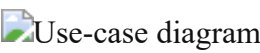


Figure 9.1. Use-case diagram. Highlighted ellipses are the Review 1 demonstration path. Dashed ellipses are designed for later sprints.

9.1 Detailed use case — UC-04 Request to Rent

Table 9.1 — UC-04

Field	Description
Actor	Tenant (primary); Landlord (secondary, later in UC-05)
Precondition	Tenant authenticated; listing exists and available=true
Main flow	1. Tenant opens /listings/:id. 2. Chooses Request to Rent. 3. System checks login and uniqueness. 4. Inserts rental_requests (kind=rent, pending). 5. Tenant dashboard lists the request.
Alternative	Not logged in → /login then return. Duplicate → return existing row. Listing not available → disable button and explain.
Postcondition	One pending rent request visible to the listing’s landlord inbox
FR	FR-04, FR-06

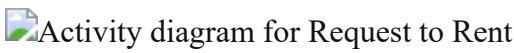


Figure 9.2. Activity diagram for the tenant Request to Rent path used in the live demonstration.

UC-05 (accept/decline) is specified by Figure 12.1. UC-01 (login) is specified by Figure 12.3. DFD at context level: tenants and landlords send listing and request data into the SPA; the SPA writes to PostgreSQL via Supabase; the database returns listing catalogues and request states. Payment and AI processes are external entities on the context diagram until those sprints start.

10. System Architecture

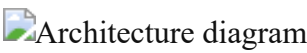


Figure 10.1. Layered architecture for Review 1, with payment and AI shown as planned adapters.

The SPA owns navigation, forms and role layouts. Adapters in frontend/src/api hide whether data comes from mock localStorage or from Supabase. For the oral exam, VITE_USE MOCK=false and the Supabase URL/anon key are set so that every successful Request to Rent is a PostgreSQL insert. Supabase Auth issues JWTs; RLS enforces Table 8.1. This is still a backend: Auth, PostgREST and Postgres run on the Supabase project, which the proposal already named as the data and auth platform.

Rationale. A two-student team cannot operate Kubernetes and still finish AI and PayWay. Managed Postgres + RLS is the feasible way to show a real database on 27 August while keeping React screens

that already exist.

11. Database Design



Figure 11.1. ER diagram. Solid boxes are Review 1 foundation tables. Dashed boxes are schema-ready for later FRs.

Table 11.1 — Data dictionary (core)

Table	Column	Type	Constraint / note
profiles	id	uuid	PK, FK to auth.users
profiles	role	text	CHECK IN ('tenant','landlord','admin')
profiles	email	text	UNIQUE NOT NULL
properties	id	uuid	PK
properties	landlord_id	uuid	FK profiles NOT NULL
properties	price	numeric(12,2)	> 0
properties	available	boolean	DEFAULT true
rental_requests	id	uuid	PK
rental_requests	kind	text	rent viewing
rental_requests	status	text	pending accepted declined
rental_requests	—	—	UNIQUE (property_id, tenant_id, kind)
contracts	request_id	uuid	FK rental_requests UNIQUE
saved_properties	(tenant_id, property_id)	uuid, uuid	Composite PK

Relationships: one landlord has many properties; one property has many requests; one accepted rent request has one contract; many-to-many save between tenants and properties. When a rent request is accepted, a database function or application transaction updates the request, inserts the contract, and sets `properties.available = false` (NFR-04).

12. Detailed Design Diagrams



Figure 12.3. Login sequence. Role is loaded from profiles.

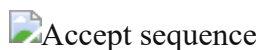


Figure 12.1. Accept-request sequence used in the live demonstration.

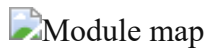


Figure 12.4. Frontend modules and the mock versus Supabase split.



Figure 12.2. Deployment: workstation demo now; Vercel later.

State of a rental request: pending → accepted | declined (terminal for that row). Listing availability: true → false on accept; admin or landlord may set true again when the contract ends (later sprint). Component diagram: Presentation (pages), Application (context + api), Infrastructure (Supabase). No additional process server is required for Review 1.

13. UI/UX Design

The interface is a responsive web app (not a native shell). Public chrome uses a top navigation bar (Home, Rent, Discover, About, Login). Authenticated chrome uses a role-specific sidebar. Filenames follow the project rule: `TenantOverview.jsx`, `LandlordPayments.jsx`, `AdminTenants.jsx` — never a shared `Overview.jsx` in two role folders.

Table 13.1 — Key screens and workflows

Screen	Route	Purpose in Review 1
Home	/	Value proposition and entry to Rent
Rent / listings	/rent	Search, filters, map (Leaflet)
Property detail	/listings/:id	Request to Rent; request viewing; save
Login / Register	/login, /register	Auth; register tenant or landlord only
Tenant requests	/dashboard/tenant/requests	See pending / accepted
Landlord requests	/dashboard/landlord/requests	Accept / decline
Landlord listings	/dashboard/landlord/listings	CRUD
Admin landlords	/dashboard/admin/landlords	Approve / suspend

Wireframes for Review 1 are the implemented screens (high-fidelity). The live demonstration uses those screens rather than a separate Figma click-dummy, which avoids two sources of truth. Annotated screenshot strips for the demo path are in Section 19; additional printouts can be appended after dress rehearsal.

Usability rules already in the UI: primary actions persist (toasts), role badge on dashboards, disabled duplicate request, and “coming soon” for Settings/Help rather than dead buttons. Login still shows a role picker in mock mode; that picker must be hidden when the Supabase adapter is on (FR-01a).

14. API and Integration Design

The frontend contract is documented in the project API contract (restore from git if missing in the working tree). Review 1 maps P0 operations onto Supabase as follows.

Table 14.1 — Endpoint to data-store mapping

Contract path	Auth	Supabase operation	Errors
POST /api/auth/register	Public	auth.signUp + insert profiles	400 validation; 409 email exists
POST /api/auth/login	Public	signInWithPassword + select profile	401 invalid
GET /api/auth/me	JWT	select profiles where id = uid	401
GET /api/properties	Public	select properties (available or all public)	500
POST /api/properties	Landlord	insert; landlord_id = uid	401, 403
POST /api/requests	Tenant	insert rental_requests	401, 409 duplicate, 400 unavailable
GET /api/requests/mine	Tenant	select where tenant_id = uid	401
GET /api/requests/inbox	Landlord	select requests join properties on landlord_id = uid	401, 403
PATCH /api/requests/:id	Landlord	update status + contract + available	403, 409 conflict

The SPA may call these as REST later (Express on port 5000) or as equivalent Supabase JS calls that keep the same JSON field names. Review 1 prefers the Supabase JS path to reduce moving parts. CORS is not required for the anon key from localhost; if Express is introduced, CORS must allow `http://localhost:5173`.

External services (designed, not all live): Leaflet/OSM (live in UI); ABA PayWay or Stripe sandbox (FR-05); OpenAI or Gemini (FR-09) with a rule-based fallback (same city and price band). Failures return a user-visible message; they must not leave half-written request rows (transaction or compensating update).

15. Security and Privacy Design

- **Authentication.** Supabase Auth, email/password, JWT stored as the existing session shape { user, token } under the documented localStorage key only as a client cache of a server session.
- **Authorisation.** RequireRole for routes; RLS for tables. Landlords cannot update another landlord's listing. Tenants cannot accept requests.

- **Admin.** Seeded account only. Register UI offers tenant and landlord. Mock role picker is development-only.
- **Validation.** Required fields, email format, price > 0, status enum, UUID identifiers. Server (RLS + check constraints) is authoritative.
- **Data protection.** No full card numbers. Demo personal data is synthetic. Production would add HTTPS only, which Supabase already uses.
- **Configuration.** Anon key is a publishable client key with RLS; service role key never ships in the Vite bundle.
- **Privacy.** Collect name, email, optional phone/Telegram. Purpose: account and rental coordination. Retention: project duration plus thesis archive. Cambodia's personal-data expectations are treated as data minimisation and no unnecessary ID scans (out of scope: title registry).

16. Technology Setup and Repository

Table 16.1 — Versions

Layer	Choice	Version / note
Frontend	React + Vite + Tailwind	React 19, Vite 6, Tailwind 4, React Router 7
Auth / DB	Supabase (PostgreSQL 15+)	Hosted free tier
Maps	Leaflet + OpenStreetMap	leaflet 1.9, react-leaflet 5
PDF invoices (UI)	jsPDF	Already in the SPA
VCS	Git / GitHub	https://github.com/Rithmunny/Property-Rental-System.git
IDE	Visual Studio Code / Cursor	—

Run (frontend): `cd frontend && npm install && npm run dev` → <http://localhost:5173>

Environment: copy `frontend/.env.example`. For the Review 1 database demo set `VITE_USE MOCK=false`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`. SQL lives in `supabase/migrations/` (to be applied on the shared project).

Branches. Feature branches for UI work; schema on a backend/schema branch that Phanith owns. Main is protected by review between the two students when possible. Root README must describe setup, demo accounts and the request-to-rent script (the README was removed from GitHub in commit 72833ad and is restored for this review).

Contribution rule for the panel: Rithmunny's account already has UI commits. Phanith must push at least the migration files, RLS, seed SQL and a short backend note so the repository shows both names. Shared documents (this PDF, slides) are group work.

17. Updated Work Plan and Risk Register

The Review 0 Gantt placed Review 1 in late September. Official student guidelines set Review 1 on **27 August 2026**. The plan below is the correction. Final delivery remains **7 November 2026**.

Table 17.1 — Revised schedule

Task	Owner	Start	End
Review 1 document, diagrams, slides	Both	20 Aug 2026	26 Aug 2026
Supabase schema, RLS, seed, demo users	Phanith	20 Aug 2026	25 Aug 2026
Wire auth / properties / requests adapters	Rithmunny	21 Aug 2026	26 Aug 2026
Review 1 oral + live demo	Both	27 Aug 2026	27 Aug 2026
Listing CRUD + search on live DB; validation	Both	28 Aug 2026	10 Sep 2026
Review 2 core (~50–60%)	Both	—	11 Sep 2026
Sandbox payment	Phanith	12 Sep 2026	30 Sep 2026
AI recommendations + fallback	Both	12 Sep 2026	8 Oct 2026
Notifications, reviews, analytics	Rithmunny	20 Sep 2026	8 Oct 2026
Review 3 pre-final	Both	—	9 Oct 2026
Testing, manuals, thesis 5–9	Both	10 Oct 2026	25 Oct 2026
Final presentation	Both	26–30 Oct 2026	30 Oct 2026
Final thesis package	Both	31 Oct 2026	7 Nov 2026

Table 17.2 — Risk register (updated)

Risk	Impact	Likelihood	Mitigation
Review 1 demo still on localStorage only	High (panel fail on prototype)	Medium	Prioritise three tables + auth; show Table Editor live
Only one GitHub account has commits	High (individual marks)	Medium	Phanith commits SQL this week
PayWay KYC delay	Medium	Medium	Stripe sandbox fallback (already in proposal)
AI quality on tiny seed set	Medium	High	Rule-based fallback; 70% relevance tested later
Scope creep (chat, maps, invoices)	High	Medium	FR MoSCoW; supervisor for new Must items
Campus network blocks Supabase	Medium	Low	Phone hotspot; screenshot/video backup (does not replace live system)
Personal data in screenshots	Medium	Low	Synthetic demo accounts only

18. Updated Thesis Progress (Chapters 1–4)

Chapter numbering follows the Batch 11 thesis handbook. Design material in Sections 9–15 of this package will become Chapter 5 after Review 1. Text below is the working draft for Chapters 1–4, using the same terms as the rest of this document.

Chapter 1 — Introduction (draft)

Long-term rental in Phnom Penh is still coordinated largely through social posts and messaging applications even though the city’s housing stock and classified inventory are large (Bamboo Routes, 2026; Realestate.com.kh, 2026). Landlords lack a single register of availability and applicants; tenants repeat the same questions to many owners. This thesis reports the design and development of a web-based Property Rental System that provides listing management, a rental-request workflow, role-based dashboards and, in later iterations, sandbox payment and AI-assisted recommendations.

Problem. Manual, fragmented coordination causes delay, double allocation and weak comparison.

Aim. Deliver a working system that centralises those jobs for tenant, landlord and administrator.

Objectives are the seven statements in Section 3.3. **Scope** is the in-scope list in Section 3.4; exclusions include live payments, legal contract generation, land-title APIs and native apps.

Significance. The work is a full-stack academic product with booking-integrity logic rather than a brochure website. **Organisation.** Chapter 2 reviews systems and literature; Chapter 3 specifies requirements; Chapter 4 states methods; later chapters will cover design, implementation, testing and evaluation.

Chapter 2 — Literature Review and Existing Systems (draft)

The comparison in Section 4 is the Chapter 2 core: Realestate.com.kh versus informal Facebook/Telegram versus short-stay booking platforms. The gap is operational closure (request, accept, record, later pay) on a Cambodian long-term rental product. Supporting concepts are RBAC, relational integrity for occupancy, BaaS, and sandbox payment ethics. Sources are listed in Section 20 and will be expanded with additional peer-reviewed PMS and recommender papers before the final thesis.

Chapter 3 — Requirements Analysis (draft)

Stakeholders and elicitation are Sections 5 and 8. Functional and non-functional requirements are Tables 6.1 and 7.1. Use cases and acceptance criteria are Sections 8–9. Prioritisation uses MoSCoW with an explicit Review 1 column so that the oral exam is not judged against unpaid PayWay work. Traceability is ID → use case → table → screen.

Chapter 4 — Methodology (draft)

The group uses Agile/Scrum with two-week sprints and a GitHub-centred backlog. Early sprints build authentication, listings and the request state machine; payment and AI follow, as in the approved feasibility text. Elicitation mixed observation, informal discussion and a questionnaire (Appendix A). Ethics: synthetic listings; no real deposits; no confidential documents uploaded to public AI tools. Quality controls: requirement quality checklist, RLS, adapter tests against the API contract, and rehearsal of the demo script. Risks and schedule are Section 17. Tools are Table 16.1.

19. Prototype Evidence

The current repository contains a complete role-based UI. Data access is switched by `VITE_USE MOCK`. For Review 1 the group must run the **database path** of the same UI. The demonstration script below is the minimum end-to-end workflow required by the guidelines (login → valid input → backend/Auth+Postgres → stored row → displayed result).

Table 19.1 — Live demonstration script

Step	Actor	Action	Expected result
0	Both	Show VS Code + GitHub remote + <code>npm run dev</code>	<code>http://localhost:5173</code>
1	Tenant	Log in with the tenant demo account (no admin picker)	Tenant dashboard
2	Tenant	Open Rent, open a listing owned by the demo landlord, Request to Rent	Pending confirmation in UI
3	Phanith	Open Supabase Table Editor → <code>rental_requests</code>	New pending row
4	Landlord	Log in, Requests, Accept	Status accepted; listing unavailable
5	Tenant	Refresh Requests	accepted (and contract stub if enabled)
6	Optional	Wrong password / tenant opens landlord URL	Error / redirect (NFR-01)

Test accounts (Classroom private comment; change passwords before a public repo README if needed):

- Tenant: `tenant.demo@prs.local` / (shared demo password)
- Landlord: `landlord.demo@prs.local` / (shared demo password) — owns the listing used in step 2
- Admin: `admin.demo@prs.local` / (shared demo password)

Honest status at document compile time. Screens and the mock adapter already perform steps 1–2 and 4–5 in the browser. The Review 1 pass criterion is those same steps with rows visible in PostgreSQL (step 3). Screenshot plates should be inserted after the dress rehearsal: (a) login, (b) property detail, (c) Table Editor, (d) landlord accept, (e) tenant accepted status. Until those photos are pasted, Figure 9.2 and Figure 12.1 are the design evidence of the same workflow.

Development environment evidence: Node 22, Vite 6, repository URL in Table 1.1, SQL migrations folder, and both students able to explain Figure 12.4.

20. References and Appendices

References (APA 7th)

Bamboo Routes. (2026). *Phnom Penh real estate market: Statistics and outlook*. <https://bambooroutes.com/blogs/news/phnom-penh-real-estate-market>

Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security*, 4(3), 224–274. <https://doi.org/10.1145/501978.501980>

Global Property Guide. (2026). *Cambodia's residential real estate market analysis 2026*. <https://www.globalpropertyguide.com/asia/cambodia/price-history>

Khmer Times. (2025). Key takeaways from Knight Frank's 2025 H1 real estate report – Phnom Penh. <https://www.khmertimeskh.com/501750031/>

MDN Web Docs. (2026). *SPA (Single-page application)*. <https://developer.mozilla.org/en-US/docs/Glossary/SPA>

OpenAI. (2026). *API documentation*. <https://platform.openai.com/docs>

Realestate.com.kh. (2026). *Cambodia property for sale & rent*. <https://www.realestate.com.kh/>

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

Stripe. (2026). *Testing*. <https://docs.stripe.com/testing>

Supabase. (2026). *Supabase documentation*. <https://supabase.com/docs>

Kirirom Institute of Technology. (2026a). *Batch 11 Review 0 student guidelines*.

Kirirom Institute of Technology. (2026b). *Batch 11 Review 1 student guidelines*.

Kirirom Institute of Technology. (2026c). *Final-year software development project and thesis student handbook* (Batch 11, June 2026).

Appendix A — Questionnaire instrument

Title: Property rental search and booking (academic, B11-G06). Time: 8–10 minutes. Consent: voluntary, anonymous, for the final-year project only.

1. Role: tenant / landlord / both.
2. How do you usually find or advertise a rental? (Facebook, Telegram, website, agent, other).
3. About how many separate chats or calls does a typical search take?
4. Have you experienced a listing that was already taken after you agreed? Yes / No / Not sure.
5. Would you submit a “request to rent” inside a website if you could see the status (pending/accepted)? Yes / Maybe / No.
6. How important is seeing the same unit's price, photos and map together? 1–5.
7. Would you pay a deposit through a known sandbox or test payment page for a student project demonstration? Yes / No.
8. As a landlord, what is hardest: photos, answering chats, tracking applicants, collecting rent, other.
9. Which filters matter most? Location, price, type, furnished, parking (rank top 3).
10. Would personalised suggestions help if they were clearly marked as suggestions? Yes / No.
11. What would stop you from using such a site? Trust, fees, Khmer language, mobile, other.
12. Any comment.

Appendix B — Interview / discussion protocol

Opening: purpose, no recording without consent, synthetic follow-up data. Prompts: walk me through the last time you rented or listed; where it failed; what “booked” means to you; who must approve. Closing: thank you; no payment required. Notes stored without full names in Appendix C as themes only (Table 5.2).

Appendix C — Traceability extract

Table C.1 — Sample traceability

FR	Use case	Table / API	Screen	Test idea
FR-01a	UC-01	profiles / login	Login.jsx	Wrong password; role from DB
FR-03	UC-03	properties	Rent.jsx	Filter city + price
FR-04	UC-04	POST requests	PropertyDetail.jsx	Row in rental_requests
FR-04a	UC-05	PATCH + contracts	LandlordRequests.jsx	available=false
FR-06	—	RequireRole	App.jsx routes	Cross-role URL
FR-11	UC-07	admin landlords	AdminLandlords.jsx	Suspend persists

Appendix D — SQL sketch (Review 1)

```
-- profiles triggered from auth.users
-- properties(landlord_id references profiles)
-- rental_requests(property_id, tenant_id, kind, status)
-- unique(property_id, tenant_id, kind)
-- contracts(request_id unique)
-- RLS: authenticated read of available properties;
-- tenants insert/select own requests;
-- landlords update requests for owned properties;
-- no public insert on profiles.role = admin
```

Full migration files in the repository are the implementation of this sketch.

Appendix E — Presentation alignment

Slides 1–14 of the Review 1 deck map onto Sections 1–3, 2, 3, 4, 5, 8, 6–7, 9, 10, 11–12, 13+19, 15–16, 16–17, and 17+19 of this document respectively. Both students speak. Demo follows Table 19.1 without reading paragraphs from slides.